

REPORT

NO	CWE-ID	VULNERABILITY	SEVERITY
1	CWE-89	LOGIN PAGE BYPASS USING SQL INJECTION	CRITICAL
2	CWE-352	CROSS SITE REQUEST FORGERY (CSRF)	HIGH
3	CWE-79	REFLECTED CROSS SITE SCRIPTING	MEDIUM
4	CWE-79	STORED CROSS SITE SCRIPTING	MEDIUM
5	CWE-209	APPLICATION VERSION DISCLOSURE via HTTP RESPONSE	MEDIUM
6	CWE-200	DIRECT IP ADDRESS ACCESS	LOW
7	CWE-693	MISSING CSP HEADER	LOW
8	CWE-693	CLICKJACKING	HIGH
9	CWE-840	BUSINESS LOGIC ERROR	HIGH
10		INSECURE COOKIE CONFIGURATION – MISSING SECURITY ATTRIBUTES	MEDIUM
11	CWE-384	SESSION FIXATION	HIGH

INFORMATION GATHERING:

Ip address: 65.61.137.117, I found the IP address using <https://whois.domaintools.com/testfire.net>

LOGIN PAGE BYPASS USING SQL INJECTION

- CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Description →

vulnerability that occurs when an attacker is able to manipulate a web application's SQL queries by injecting malicious input. This can allow the attacker to bypass authentication mechanisms , such as login pages, and potentially gain access to sensitive data, including admin credentials.

- CVSS Version 4.0 Calculator

CVSS
Common Vulnerability Scoring System Version 4.0 Calculator

CVSS-4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N [Reset]

CVSS v4.0 Score: 9.3 / Critical

Attack Complexity (AC): Low (L) | High (H)

Attack Requirements (AR): None (N) | Present (P)

Privileges Required (PR): None (N) | Low (L) | High (H)

User Interaction (UI): None (N) | Passive (P) | Active (A)

Vulnerable System Impact Metrics

Confidentiality (VC): High (H) | Low (L) | None (N)

Integrity (VI): High (H) | Low (L) | None (N)

Availability (VA): High (H) | Low (L) | None (N)

Steps to reproduce →

First Step I will Go to <http://testfire.net/login.jsp> and open login function Enter username 'OR 1=1-- (or) admin'-- and enter any value in the password field and then click on login and I login as administrator

PoC (Proof of Concept) →

AltoroMutual

Sign In | Contact Us | Feedback | Search

Online Banking Login

Username: OR 1=1--

Password: *****

Login

This web application is open source! Get your copy from GitHub and take advantage of advanced features

The Altoro website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to <https://www.hcl-software.com/appscan/>.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.

AltoroMutual

Sign Off | Contact Us | Feedback | Search

MY ACCOUNT

Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details: 800000 Corporate GO

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

Click [here](#) to apply.

This web application is open source! Get your copy from GitHub and take advantage of advanced features

The Altoro website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to <https://www.hcl-software.com/appscan/>.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.

IMPACT →

SQL INJECTION allows the attacker to bypass login page and take

admin credentials, Modify or delete database records

REMEDIATION →

1. Implement parameterized queries / prepared statements
2. Enforce strict server-side input validation
3. Avoid dynamic SQL query construction

CROSS SITE REQUEST FORGERY (CSRF)

- CWE-352: Cross-Site Request Forgery (CSRF)
- CVSS Version 4.0 Calculator

The screenshot displays the CVSS Version 4.0 Calculator interface. On the left is a sidebar with navigation links: Calculator, Specification Document, User Guide, Examples, Frequently Asked Questions, CVSS v4.0 Documentation & Resources, CVSS v3.1 Archive, CVSS v3.0 Archive, CVSS v2 Archive, CVSS v1 Archive, JSON & XML Data, Representations, CVSS On-Line Training Course, and Identity & logo usage. The main area is titled 'Common Vulnerability Scoring System Version 4.0 Calculator'. It features a text input field for the CVSS string 'CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:P/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N' and a 'Reset' button. Below this, the 'CVSS v4.0 Score: 7.6 / High' is displayed. The interface is divided into sections for 'Attack Complexity (AC)', 'Attack Requirements (AT)', 'Privileges Required (PR)', 'User Interaction (UI)', and 'Vulnerable System Impact Metrics'. Each section contains radio buttons for different severity levels. For example, in the 'Attack Complexity (AC)' section, 'Low (L)' is selected. In the 'Vulnerable System Impact Metrics' section, 'High (H)' is selected for Confidentiality (VC), Integrity (VI), and Availability (VA).

Description →

The application is vulnerable to **Cross-Site Request Forgery (CSRF)**.

The affected endpoint does not implement adequate CSRF protection mechanisms, allowing attackers to **force authenticated users to perform unintended actions** without their knowledge.

Steps to reproduce →

1. First I select a request of transfer money in Burp Suite community edition that i want to test or exploit. From the right-click context menu, and copy the URL go to the exploit server, paste exploit HTML into the "Body" section, and click "Store" and delivery to victim.

- To verify the behaviour, I use the 'View exploit' option on myself in the lab setup. And the transaction page appears, it confirms the exploit simulation worked.

PoC (Proof of Concept) →

The screenshot displays the Burp Suite interface, which is a web security tool. The top navigation bar includes tabs for Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Learn. The main workspace is divided into three panels: Request, Response, and Inspector.

Request Panel: Shows a GET request to `/bank/transfer.jsp` on `demo.testfire.net`. The request includes a session ID and various headers like `Accept-Language`, `User-Agent`, and `Accept-Encoding`.

Response Panel: Shows the corresponding HTTP 200 OK response from the server, including headers like `Server: Apache-Coyote/1.1` and `Content-Type: text/html; charset=utf-8`.

Inspector Panel: Displays the HTML structure of the response, showing the `<html>` tag with attributes like `xmlns` and `lang`.

PoC Exploit Simulation: Below the main interface, a PoC exploit is shown. It includes a file path `/exploit` and a head section with the following content:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

The body of the exploit is a JavaScript snippet that simulates a form submission:

```
<html>
<body>
  <form action="https://https://demo.testfire.net/bank/transfer.jsp?website.com/email/change" method="POST">
    <input type="hidden" name="email" value="pwne@evil-user.net" />
  </form>
  <script>
    document.forms[0].submit();
  </script>
</body>
</html>
```

At the bottom of the PoC section, there are four buttons: **Store**, **View exploit**, **Deliver exploit to victim**, and **Access log**.

AltoroMutual

Sign Off | Contact Us | Feedback | Search [] Go

MY ACCOUNT PERSONAL SMALL BUSINESS INSIDE ALTORO MUTUAL

I WANT TO ...

- View Account Summary
- View Recent Transactions
- Transfer Funds
- Search News Articles
- Customize Site Language

ADMINISTRATION

- Edit Users

Transfer Funds

From Account: 800000 Corporate

To Account: 800000 Corporate

Amount to Transfer: []

Transfer Money

Privacy Policy | Security Statement | Server Status Check | BEST API | © 2025 Altoro Mutual, Inc.

This web application is open source! Get your copy from GitHub and take advantage of advanced features

The Altoro website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to <https://www.hcl-software.com/agoscan/>.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2025, HCL Technologies, Ltd., All rights reserved.

IMPACT →

Successful exploitation could allow an attacker to:

- Change user passwords
- Modify user profile details
- Perform unauthorized transactions
- Execute privileged actions as the victim user

REMEDIATION →

- Implement **anti-CSRF tokens**
- Validate CSRF tokens on all state-changing requests
- Enforce **Same Site=Strict/Lax** on session cookies

REFLECTED CROSS SITE SCRIPTING

- **CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')**
- **CVSS** Version 4.0 Calculator

CVSS
Common Vulnerability Scoring System Version 4.0 Calculator

CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:P/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N

CVSS v4.0 Score: 5.1 / Medium

Attack Complexity (AC):

Attack Requirements (AT):

Privileges Required (PR):

User Interaction (UI):

Vulnerable System Impact Metrics

Confidentiality (VC):

Integrity (VI):

Description →

The application is vulnerable to Reflected Cross-Site Scripting (XSS) due to improper sanitization and encoding of user-supplied input in the search parameter. An attacker can inject malicious JavaScript code which is reflected in the server response and executed in the victim's browser.

Steps to reproduce →

- Navigate to the search page
- Enter the following payload into the search input: `<script>alert(1)</script>`
- Submit the search request
- Observe the JavaScript alert execution

PoC (Proof of Concept) →

AltoroMutual

Sign Off | Contact Us | Feedback | Search

ACCOUNT

VAL
[Deposit Product](#)
[Checking](#)
[Loan Products](#)
[Cards](#)
[Investments & Insurance](#)
[Other Services](#)

BUSINESS
[Deposit Products](#)
[Lending Services](#)
[Cards](#)
[Insurance](#)
[Retirement](#)

PERSONAL

Online Banking with FREE Online Bill Pay
 No stamps, envelopes, or checks to write give you more time to spend on the things you enjoy.

Real Estate Financing
 Fast Simple Professional Whether you are preparing to buy

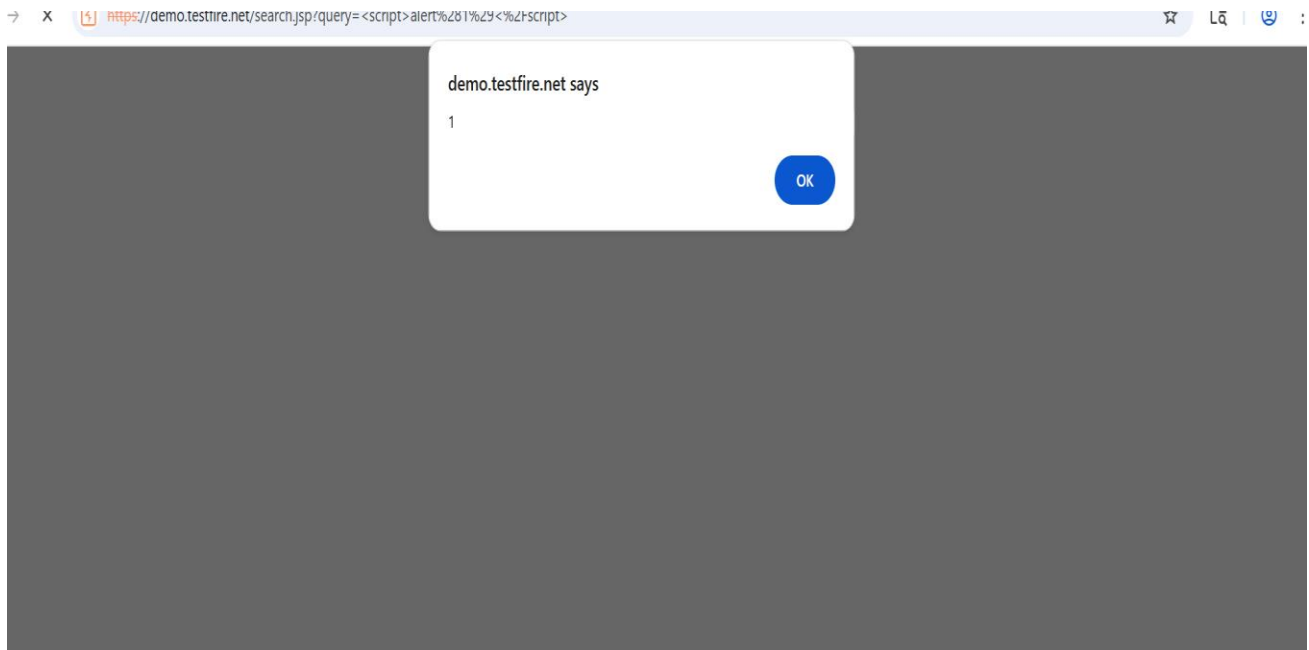
SMALL BUSINESS

Business Credit Cards
 You're always looking for ways to improve your company's bottom line. You want to be informed, improve efficiency and control expenses. Now, you can do it all - with a business credit

INSIDE ALTORO MUTUAL

Privacy and Security
 The 2000 employees of Altoro Mutual are dedicated to protecting your [privacy](#) and [security](#). We pledge to provide you with the information and resources that you need to help secure your information and keep it confidential. This is our promise.

DEMO SITE ONLY



IMPACT →

This vulnerability allow attacker to execute JavaScript targeting the end users across the web application.

REMEDIATION →

Enable **Content Security Policy (CSP)**

STORED CROSS SITE SCRIPTING →

- **CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-Site Scripting')**
- **CVSS Version 4.0 Calculator**

The screenshot displays the CVSS 4.0 Calculator interface. On the left is a navigation menu with links like 'Vulnerability Scoring (CVSS-SIG)', 'Documentation', 'Help', 'Recently Asked Questions', '4.0 Documentation & Resources', and various archives. The main area is titled 'Common Vulnerability Scoring System Version 4.0 Calculator'. It shows a CVSS string: 'CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:P/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N' and a 'Reset' button. Below this, the 'CVSS v4.0 Score: 7 / High' is displayed. The interface is divided into sections for 'Attack Vector (AV)', 'Attack Complexity (AC)', 'Attack Requirements (AT)', 'Privileges Required (PR)', 'User Interaction (UI)', and 'Vulnerable System Impact Metrics'. Each section contains buttons for different values, with some values highlighted in green to indicate the current selection. For example, in the 'Attack Vector' section, 'Network (N)' is selected. In the 'Vulnerable System Impact Metrics' section, 'High (H)' for Confidentiality (VC) and 'Low (L)' for Integrity (VI) are selected.

Section	Value
Attack Vector (AV)	Network (N)
Attack Complexity (AC)	Low (L)
Attack Requirements (AT)	None (N)
Privileges Required (PR)	Low (L)
User Interaction (UI)	Passive (P)
Vulnerable System Impact Metrics	Confidentiality (VC): High (H)
Vulnerable System Impact Metrics	Integrity (VI): Low (L)
Vulnerable System Impact Metrics	Availability (VA): None (N)

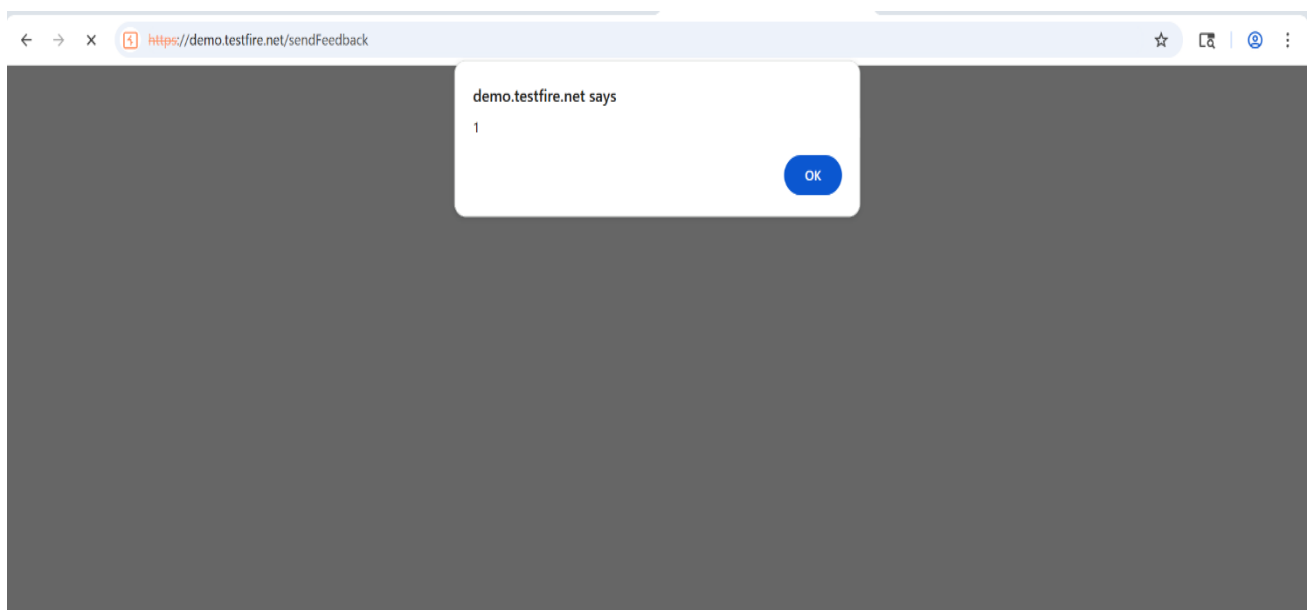
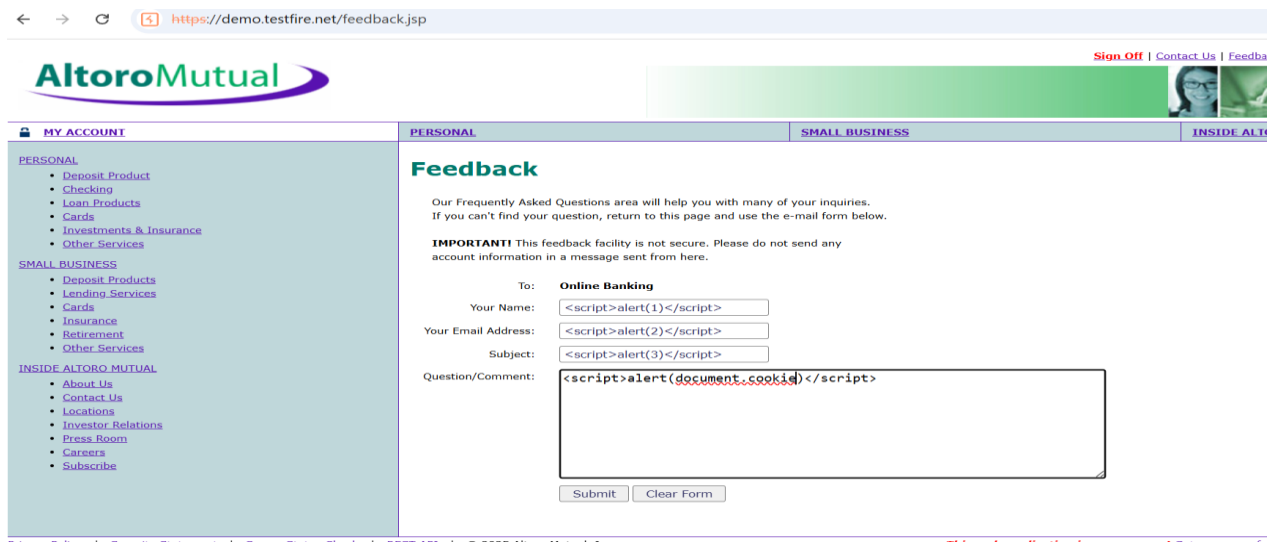
Description →

The application is vulnerable to Stored Cross-Site Scripting (XSS) due to improper sanitization and output encoding of user-supplied input submitted through an online form. The malicious script is stored in the backend database and executed whenever another user or administrator views the affected content.

Steps to reproduce →

- Navigate to the online form
- Enter the following payload in a text field: `<script>alert(1)</script>`
- Submit the form
- Observe JavaScript execution

PoC (Proof of Concept) →



IMPACT →

Exploitation of this vulnerability may allow attackers to:

- Execute arbitrary JavaScript in victims' browsers
- Steal session cookies or authentication tokens
- Perform actions as logged-in users
- Deliver malware or phishing attacks

REMEDIATION →

- Apply **server-side input validation**
- Implement **context-aware output encoding**
- Implement **Content Security Policy (CSP)**

INFORMATION DISCLOSURE

Application Version Disclosure via HTTP Response

- **CWE-209 – Information Exposure Through an Error Message**

CVSS Version 4.0 Calculator

Common Vulnerability Scoring System (CVSS-516)

- Calculator
- Specification Document
- User Guide
- Examples
- Frequently Asked Questions
- CVSS v4.0 Documentation & Resources
- CVSS v3.1 Archive
- CVSS v3.0 Archive
- CVSS v2 Archive
- CVSS v1 Archive
- JSON & XML Data
- Representations
- CVSS On-Line Training Course
- Identity & logo usage

CVSS Common Vulnerability Scoring System Version 4.0 Calculator

CVSS-4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

CVSS v4.0 Score: **6.9 / Medium** ⊕

CVSS v4.0 Base Score

Network (N)	Adjacent (A)	Local (L)	Physical (P)
Attack Complexity (AC):			
Low (L)	High (H)		
Attack Requirements (AT):			
None (N)	Present (P)		
Privileges Required (PR):			
None (N)	Low (L)	High (H)	
User Interaction (UI):			
None (N)	Passive (P)	Active (A)	

Vulnerable System Impact Metrics

Confidentiality (VC):	High (H)	Low (L)	None (N)
Integrity (VI):	High (H)	Low (L)	None (N)

Description →

The application discloses its underlying technology and version information through publicly accessible responses. This information may assist attackers in identifying known vulnerabilities associated with the disclosed version, thereby increasing the risk of targeted attacks.

Steps to reproduce →

- First step I go to the website
http://testfire.net/index.jsp?content=personal_deposit.htm
- Modify the content parameter by injecting a path traversal payload:
../../../../etc/passwd
- The application responds with an **Internal Server Error (HTTP 500)**.
- And The error response contains version of the server

PoC (Proof of Concept) →



Impact→

- Reveals server version information
- Enables technology fingerprinting
- Assists attackers in identifying known vulnerabilities

Remediation→

- Disable version disclosure in HTTP headers
- Configure the web server to hide or obfuscate version details

DIRECT IP ADDRESS ACCESS

CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor

Description →

The application allows access to the website directly via its IP address instead of the domain name. This behaviour may indicate improper server configuration and can allow attackers to bypass security controls that are enforced at the domain level, as Web Application Firewalls (WAF).

Steps to reproduce →

- Open a browser.
- Access the same application using the server IP address:
https://65.61.137.117/
- Observe that the application loads successfully using the IP address.

PoC (Proof of Concept) →



Impact →

- Enables bypass of domain-based security controls
- May bypass WAF or host-header security checks
- Increases the attack surface

Remediation→

- Configure the web server to deny or redirect IP-based requests
- Enforce strict Host header validation
- Apply WAF and security rules at the IP and domain level
- Redirect all IP requests to the canonical domain using HTTP 301/302

MISSING CSP HEADER

CWE-693 – Protection Mechanism Failure

Description → The application does not implement a Content Security Policy (CSP) HTTP response header. CSP is a browser-enforced security mechanism that helps mitigate Cross-Site Scripting (XSS) and other client-side injection attacks by restricting the sources from which content can be loaded.

Steps to reproduce →

1. Open a browser.
2. Access the Application using the domain name: <http://testfire.net>
3. click login functionality <https://demo.testfire.net/login.jsp>

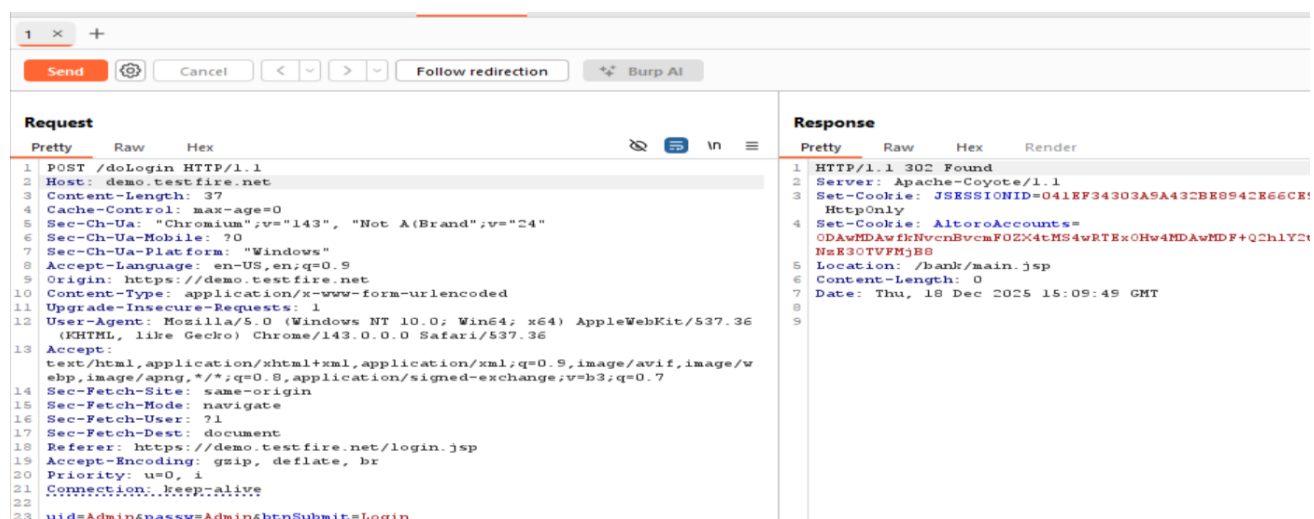
- login with credentials and capture the burp request of login page
- send the request to Repeater and observe the response there is no CSP header

Example CSP:

Content-Security-Policy:

- default-src 'self'
- script-src 'self'
- style-src 'self'
- img-src 'self' data
- frame-ancestors 'none'

PoC (Proof of Concept) →



The screenshot shows a Burp Suite interface with a request and response view. The request is a POST to /doLogin with a body containing login credentials. The response is an HTTP 302 Found status with a Location header pointing to /bank/main.jsp.

```

Request
Pretty Raw Hex
1 POST /doLogin HTTP/1.1
2 Host: demo.testfire.net
3 Content-Length: 37
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: https://demo.testfire.net
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
13 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
14 Accept:
15 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
16 Sec-Fetch-Site: same-origin
17 Sec-Fetch-Mode: navigate
18 Sec-Fetch-User: ?1
19 Sec-Fetch-Dest: document
20 Referer: https://demo.testfire.net/login.jsp
21 Accept-Encoding: gzip, deflate, br
22 Priority: u=0, i
23 Connection: keep-alive
24 uid=Admin&passw=Admin&btnSubmit=Login

Response
Pretty Raw Hex Render
1 HTTP/1.1 302 Found
2 Server: Apache-Coyote/1.1
3 Set-Cookie: JSESSIONID=041EF34303A9A432BE8942E66CE;HttpOnly
4 Set-Cookie: AltoreAccounts=0DAwMDAwfkNvcnBvcnF0ZX4tMS4wRTExOHw4MDAwMDF+Q2h1Y21NzE3OTVFMjB8;Location: /bank/main.jsp
5 Location: /bank/main.jsp
6 Content-Length: 0
7 Date: Thu, 18 Dec 2025 15:09:49 GMT
8
9

```

Impact →

- Increased exposure to Cross-Site Scripting (XSS) attacks
- Lack of browser-side protection against malicious script execution

Remediation→

- Implement a restrictive Content-Security-Policy header

CLICKJACKING

- CWE-693 – Protection Mechanism Failure

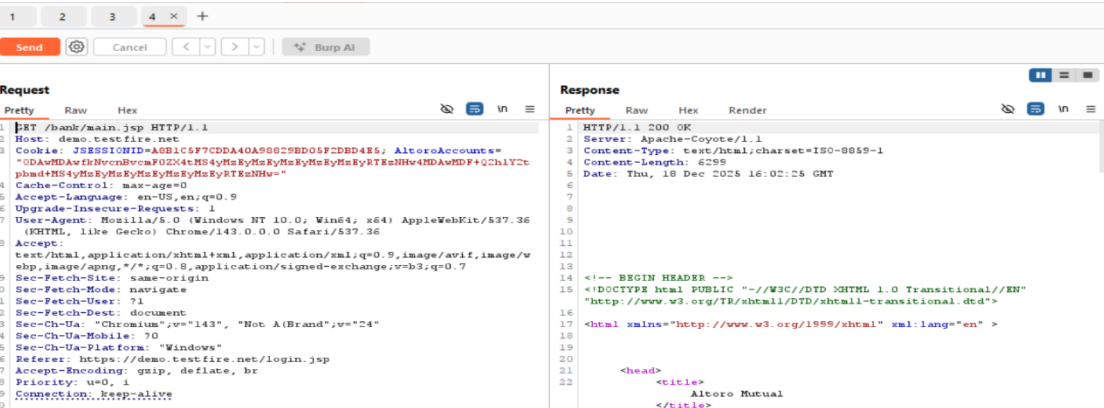
CVSS Version 4.0 Calculator

Description → The application does not implement adequate frame-busting protections, allowing it to be embedded within an `<iframe>` on a malicious website. This exposes users to clickjacking attacks, where an attacker can trick users into performing unintended actions by overlaying or disguising UI elements.

Steps to reproduce →

- First I select a request of transfer money in Burp Suite community edition that i want to test or exploit. From the right-click context menu, and copy the URL
- go to the exploit server, paste exploit HTML into the "Body" section:
- click "Store" and view exploit

PoC (Proof of Concept) →



CVSS Version 4.0 Calculator



Common Vulnerability Scoring System Version 4.0 Calculator

CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N

Reset

CVSS v4.0 Score: **8.7 / High**

Attack Vector (AV):	Network (N)	Adjacent (A)	Local (L)	Physical (P)
Attack Complexity (AC):	Low (L)	High (H)		
Attack Requirements (AT):	None (N)	Present (P)		
Privileges Required (PR):	None (N)	Low (L)	High (H)	
User Interaction (UI):	None (N)	Passive (P)	Active (A)	
Vulnerable System Impact Metrics				
Confidentiality (VC):	High (H)	Low (L)	None (N)	
Integrity (VI):	High (H)	Low (L)	None (N)	

Description → The application fails to properly enforce business logic controls during the transaction workflow. An attacker can manipulate the intended sequence of actions to perform unauthorized operations that violate business rules.

Steps to reproduce →

1. Open a browser.
2. Access the Application using the domain name: <http://testfire.net>
3. click login functionality <https://demo.testfire.net/login.jsp>
4. login with credentials
5. go to transfer funds and check from account (800000 corporate) and change To account (800001 checking) and enter the amount beyond the limit
6. check the response it successfully transferred

PoC (Proof of Concept) →



Sign Off | Contact Us | Feedback | Search

MY ACCOUNT

PERSONAL

SMALL BUSINESS

INSIDE ALTORO MUTUAL

ANT TO ...

- View Account Summary
- View Recent Transactions
- Transfer Funds
- Search News Articles
- Customize Site Language

ADMINISTRATION

- Edit Users

Transfer Funds

From Account:

800000 Corporate

To Account:

800000 Corporate

Amount to Transfer:

Transfer Money

1.0E13 was successfully transferred from Account 800000 into Account 800001 at 12/21/25 6:18 AM.

Privacy Policy | Security Statement | Server Status Check | REST API | © 2025 Altoro Mutual, Inc.

This web application is open source! Get your copy from GitHub and

Impact→

An attacker could exploit this vulnerability to:

- Bypass mandatory business steps (e.g., payment, approval)
- Perform unauthorized actions
- Cause financial loss
- Manipulate application state
- Compromise data integrity

Remediation→

- Enforce strict server-side validation of all business rules.
- Validate workflow sequence and state transitions.
- Implement **logging and monitoring** for abnormal workflow behaviour.

INSECURE COOKIE CONFIGURATION – MISSING SECURITY ATTRIBUTES

CWE-

Description→ The application sets cookies without implementing recommended security attributes such as HttpOnly, Secure, and/or SameSite. These attributes are designed to protect cookies from client-side access, transmission over insecure channels, and cross-site request forgery attacks. Absence of these attributes increases the risk of cookie theft, session hijacking, and cross-site attacks.

Steps to reproduce →

1. Open a browser.
2. Access the Application using the domain name: <http://testfire.net>
3. click login functionality <https://demo.testfire.net/login.jsp>
4. login with credentials
5. Intercept the HTTP response after successful authentication.
6. Review the Set-Cookie header in the response.
7. observe the response there is no set cookie attributes such as HttpOnly, Secure, SameSite

PoC (Proof of Concept) →

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST	/doLogin	HTTP/1.1	1	HTTP/1.1	302	Found
2	Host:	demo.testfire.net		2	Server:	Apache-Coyote/1.1	
3	Cookie:	AltoraAccounts=		3	Set-Cookie:	AltoraAccounts=	
4		"0DAwMDAwEhNvcnBvcwF0Zk4tMS4yMzEyMzEyMzEyMzEyRTZzNHw4MDAwMDFF+QChLYTc		4		"0DAwMDAwEhNvcnBvcwF0Zk4tMS4yMzEyMzEyMzEyMzEyRTZzNHw4MDAwMDFF+QChLYTc	
5		phad+MS4yMzEyMzEyMzEyMzEyMzEyMzEyRTZzNHw4MDAwMDFF+QChLYTc		5		phad+MS4yMzEyMzEyMzEyMzEyMzEyMzEyMzEyRTZzNHw4MDAwMDFF+QChLYTc	
6	Content-Length:	37		6	Location:	/bank/main.jsp	
7	Cache-Control:	max-age=0		7	Content-Length:	0	
8	Sec-Ch-Ua:	"Chromium";v="143", "Not A(Brand";v="24"		8	Date:	Sun, 21 Dec 2025 13:09:54 GMT	
9	Sec-Ch-Ua-Mobile:	70					
10	Sec-Ch-Ua-Platform:	"Windows"					
11	Accept-Language:	en-US,en;q=0.9					
12	Origin:	https://demo.testfire.net					
13	Content-Type:	application/x-www-form-urlencoded					
14	Upgrade-Insecure-Requests:	1					
15	User-Agent:	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36					
16	(KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36						
17	Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/w					
18	ehp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7						
19	Sec-Fetch-Site:	same-origin					
20	Sec-Fetch-Mode:	navigate					
21	Sec-Fetch-User:	?1					
22	Sec-Fetch-Dest:	document					
23	Referer:	https://demo.testfire.net/login.jsp					
24	Accept-Encoding:	gzip, deflate, br					
	Priority:	u=0, i					
	Connection:	keep-alive					
	uid=Admin&passw=admin&btnSubmit=Login						

Impact→

An attacker may be able to:

- Access session cookies via client-side scripts (if HttpOnly is missing)
- Intercept cookies over unencrypted connections (if Secure is missing)
- Perform CSRF or cross-site request attacks (if SameSite is missing)

Remediation→

- Set the HttpOnly attribute on all sensitive cookies to prevent client-side access.
- Ensure the Secure attribute is enabled so cookies are only transmitted over HTTPS.
- Implement an appropriate SameSite attribute (Strict or Lax) based on application requirements.
- Review all cookies to ensure consistent secure configuration.

SESSION FIXATION

CWE-384 – Session Fixation

CVSS Version 4.0 Calculator

Common Vulnerability Scoring System Version 4.0 Calculator

CVSS:4.0/AV:N/AC:H/AT:P/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

CVSS v4.0 Score: 7.6 / High ⊕

<input checked="" type="button" value="Network (N)"/>	Adjacent (A)	Local (L)	Physical (P)
Attack Complexity (AC):			
Low (L)	<input checked="" type="button" value="High (H)"/>		
Attack Requirements (AT):			
None (N)	<input checked="" type="button" value="Present (P)"/>		
Privileges Required (PR):			
None (N)	<input checked="" type="button" value="Low (L)"/>	High (H)	
User Interaction (UI):			
<input checked="" type="button" value="None (N)"/>	Passive (P)	Active (A)	
Vulnerable System Impact Metrics			
Confidentiality (VC):			
<input checked="" type="button" value="High (H)"/>	Low (L)	None (N)	
Integrity (VI):			
<input checked="" type="button" value="High (H)"/>	Low (L)	None (N)	

Description→

The application does not regenerate a new session identifier after successful authentication. As a result, a session ID assigned prior to login remains valid after authentication. An attacker can exploit this behaviour by fixing a known session identifier for a victim and subsequently hijacking the authenticated session once the victim logs in.

Steps to reproduce →

1. Open a browser.
2. Access the Application using the domain name: <http://testfire.net>
3. click login functionality <https://demo.testfire.net/login.jsp>
4. login with credentials
5. open the burp login request and check session id after that logout the application
6. again login the application with credentials and check session id both session ids are same

PoC (Proof of Concept) →

Before login session id:

Request			R
Pretty	Raw	Hex	
1	GET /bankr/main.jsp HTTP/1.1		
2	Host: demo.testfire.net		
3	Cookie: JSESSIONID=81B01E12E77A37A84F705A67D8734A4E; AltoroAccounts="ODAwMDAwfkNvcnBvcnfOZK4tMS4wMDAxNTI5MDYzNDI3Mz1FMTNSODAwMDAxfkNoZWNaW5nfjEuMDAwMDM2OTcwNzcyMjQORTZzfa=="		
4	Cache-Control: max-age=0		
5	Accept-Language: en-US,en;q=0.9		
6	Upgrade-Insecure-Requests: 1		
7	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36		
8	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		
9	Sec-Fetch-Site: same-origin		
10	Sec-Fetch-Mode: navigate		
11	Sec-Fetch-User: ?1		
12	Sec-Fetch-Dest: document		
13	Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"		
14	Sec-Ch-Ua-Mobile: ?0		
15	Sec-Ch-Ua-Platform: "Windows"		
16	Referer: https://demo.testfire.net/login.jsp		
17	Accept-Encoding: gzip, deflate, br		
18	Priority: u=0, i		
19	Connection: keep-alive		
20			
21			

After login session id:

			P
Pretty	Raw	Hex	
1	GET /bankr/main.jsp HTTP/1.1		1
2	Host: demo.testfire.net		2
3	Cookie: JSESSIONID=81B01E12E77A37A84F705A67D8734A4E; AltoroAccounts="ODAwMDAwfkNvcnBvcnfOZK4tMS4wMDAxNTI5MDYzNDI3Mz1FMTNSODAwMDAxfkNoZWNaW5nfjEuMDAwMDM2OTcwNzcyMjQORTZzfa=="		3
4	Cache-Control: max-age=0		4
5	Accept-Language: en-US,en;q=0.9		5
6	Upgrade-Insecure-Requests: 1		6
7	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36		7
8	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		8
9	Sec-Fetch-Site: same-origin		9
10	Sec-Fetch-Mode: navigate		10
11	Sec-Fetch-User: ?1		11
12	Sec-Fetch-Dest: document		12
13	Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"		13
14	Sec-Ch-Ua-Mobile: ?0		14
15	Sec-Ch-Ua-Platform: "Windows"		15
16	Referer: https://demo.testfire.net/login.jsp		16
17	Accept-Encoding: gzip, deflate, br		17
18	Priority: u=0, i		18
19	Connection: keep-alive		19
20			20
21			21
			22
			23

Impact →

An attacker could:

- Hijack authenticated user sessions
- Gain unauthorized access to user accounts
- Perform actions on behalf of the victim
- Compromise confidentiality and integrity of user data

Remediation→

- Regenerate a new session identifier upon:
- Successful authentication
- Privilege level changes
- Invalidate the old session ID after regeneration.
- Ensure session IDs are unpredictable and securely generated.
- Avoid accepting session identifiers from untrusted sources.

